



IX CAIM
IV CAIFE



FIE
Facultad de
Ingeniería
del Ejército

FONDO
DOCENTE
CONSEJO
FACULTAD
INGENIERIAS

FoDAMI

**IV CONGRESO ARGENTINO
DE INGENIERÍA FERROVIARIA**

**IX CONGRESO ARGENTINO
DE INGENIERÍA MECÁNICA**



Introducción temprana de la mecánica computacional en cursos de electrostática y mecánica analítica

INTEGRANTES

- Víctor A. Bettachini, Dep. de Ingeniería e Investigaciones Tecnológicas, UNLaM
- Mariano A. Real, Inst. de Calidad Industrial, UNSaM – INTI
- Edgardo Palazzo, Dep. de Materias Básicas, UTN–FRA

SÍNTESIS

La ejercitación basada en la edición de código extendió el alcance de dos asignaturas:

- Modelar dispositivos industriales realistas con la mecánica analítica.
- Mayor comprensión conceptual de la electrostática al evitar matemática avanzada.

Temario

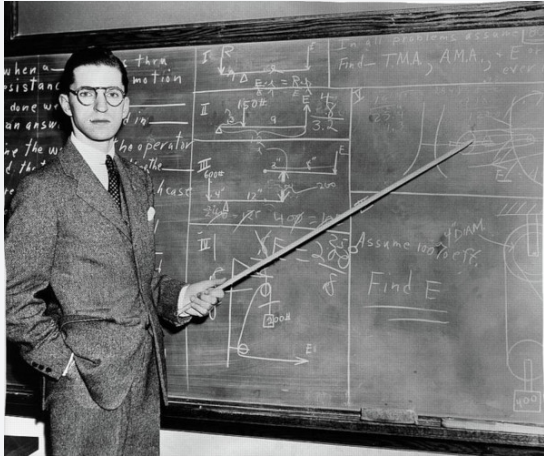
- 1 Herramientas que definen el aula
- 2 UNLaM — Mecánica analítica
- 3 UTN-FRA — Física II
- 4 Para concluir

Temario

- 
- 
- 1 Herramientas que definen el aula
 - 2 UNLaM — Mecánica analítica
 - 3 UTN-FRA — Física II
 - 4 Para concluir

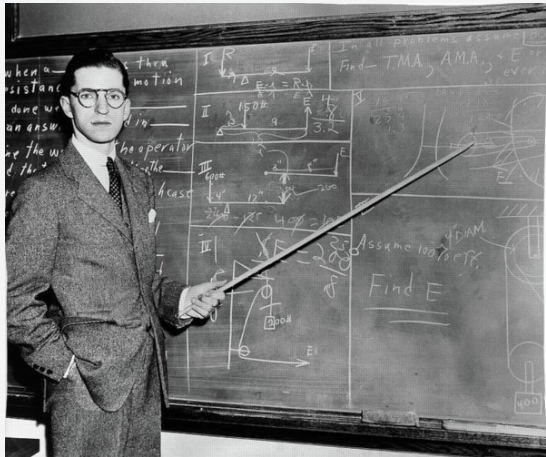
Herramientas omnipresentes

Inicio s. XIX: pizarrón y papel

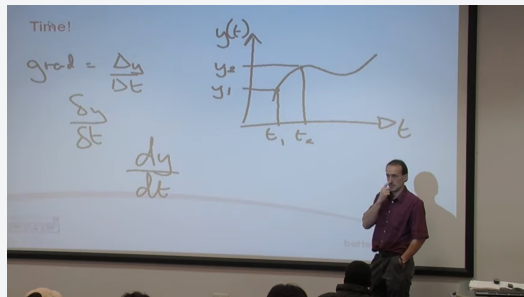


Herramientas omnipresentes

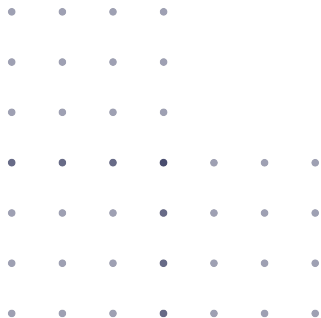
Inicio s. XIX: pizarrón y papel



Fin s. XX: computadora como proyector



Subutilización de la tecnología de la información



Subutilización de la tecnología de la información

La resistencia al cambio



Subutilización de la tecnología de la información

La resistencia al cambio



Décadas de atraso



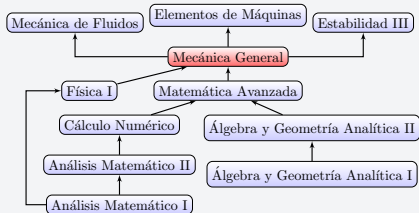
¿El miedo a la adopción tecnológica es lo único que no ha evolucionado?

Temario

- 
- 
- 1 Herramientas que definen el aula
 - 2 UNLaM — Mecánica analítica**
 - 3 UTN-FRA — Física II
 - 4 Para concluir

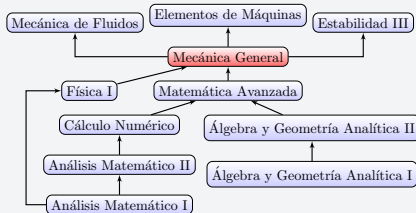
Mecánica analítica en UNLaM inició en pandemia

Puente matemática – aplicadas



Mecánica analítica en UNLaM inició en pandemia

Puente matemática – aplicadas

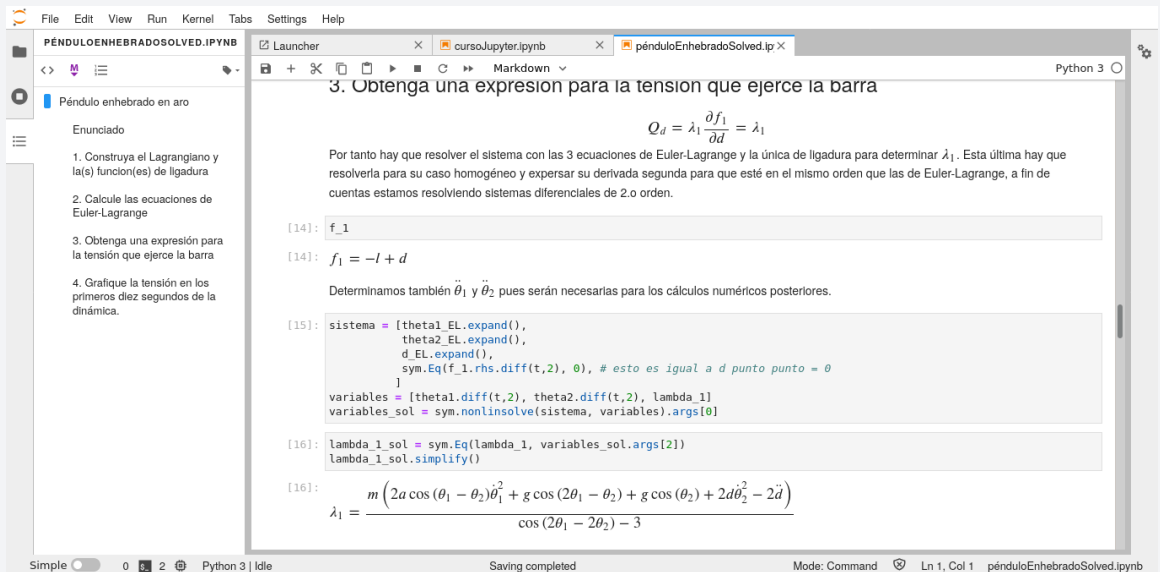


Tras una computadora... ¡Automaticémos!

```
[14]: sistemaEcuaciones = [  
      x_EL,  
      phi_EL,  
    ]  
variablesDespeje = [x.diff(t,2), phi.diff(t,2)] # despejar aceleraciones generalizadas  
variablesDespeje_sol = sym.nonlinsolve(sistemaEcuaciones, variablesDespeje ).args[0]  
  
[15]: x_pp = sym.Eq(variablesDespeje[0], variablesDespeje_sol.args[0]) # [m s-2]  
phi_pp = sym.Eq(variablesDespeje[1], variablesDespeje_sol.args[1]) # [m s-2]  
x_pp, phi_pp
```

$$[15]: \left(\begin{array}{l} \ddot{x} = \frac{-\ell m_2 \sin(\phi) + \frac{\ell m_2 (\ell m_2 \cos(\phi) \dot{\phi}^2 + g m_1 + g m_2) \sin(\phi)}{m_1 + m_2 \sin^2(\phi)}}{\ell m_2 \cos(\phi)}, \quad \ddot{\phi} = -\frac{(\ell m_2 \cos(\phi) \dot{\phi}^2 + g m_1 + g m_2) \sin(\phi)}{\ell (m_1 + m_2 \sin^2(\phi))} \end{array} \right)$$

Cuaderno Jupyter: teoría + código ejecutable



The screenshot shows a Jupyter Notebook titled "PÉNDULO ENHEBRADO SOLVED.IPYNB". The left sidebar contains a table of contents with the following items:

- Enunciado
- 1. Construya el Lagrangiano y la(s) función(es) de ligadura
- 2. Calcule las ecuaciones de Euler-Lagrange
- 3. Obtenga una expresión para la tensión que ejerce la barra
- 4. Grafique la tensión en los primeros diez segundos de la dinámica.

The main content area displays the following text and code:

3. Obtenga una expresión para la tensión que ejerce la barra

$$Q_d = \lambda_1 \frac{\partial f_1}{\partial d} = \lambda_1$$

Por tanto hay que resolver el sistema con las 3 ecuaciones de Euler-Lagrange y la única de ligadura para determinar λ_1 . Esta última hay que resolverla para su caso homogéneo y expensar su derivada segunda para que esté en el mismo orden que las de Euler-Lagrange, a fin de cuentas estamos resolviendo sistemas diferenciales de 2.º orden.

```
[14]: f_1
```

```
[14]: f_1 = -l + d
```

Determinamos también $\ddot{\theta}_1$ y $\ddot{\theta}_2$ pues serán necesarias para los cálculos numéricos posteriores.

```
[15]: sistema = [theta1_EL.expand(),
               theta2_EL.expand(),
               d_EL.expand(),
               sym.Eq(f_1.rhs.diff(t,2), 0)], # esto es igual a d punto punto = 0
        ]
variables = [theta1.diff(t,2), theta2.diff(t,2), lambda_1]
variables_sol = sym.nonlinsolve(sistema, variables).args[0]
```

```
[16]: lambda_1_sol = sym.Eq(lambda_1, variables_sol.args[2])
      lambda_1_sol.simplify()
```

```
[16]:
```

$$\lambda_1 = \frac{m \left(2a \cos(\theta_1 - \theta_2) \dot{\theta}_1^2 + g \cos(2\theta_1 - \theta_2) + g \cos(\theta_2) + 2d \ddot{\theta}_2 - 2\ddot{d} \right)}{\cos(2\theta_1 - 2\theta_2) - 3}$$

The bottom status bar indicates: Simple, 0, 2, Python 3 | Idle, Saving completed, Mode: Command, Ln 1, Col 1, pénduloEnhebradoSolved.ipynb.

Google Colaboratory: cuadernos Jupyter en la nube

- Docente puede editar y comentar cuaderno del alumno.

 07 No conservativas | ej4 ☆

Archivo Editar Ver Insertar Entorno de ejecución Herramientas Ayuda [Se editó por última vez: 3 de junio](#)

+ Código + Texto

```
[ ] # Energía potencial
m1_V = - (m1* g* (- N.y)).dot(m1_r)
# pot_k1 = unMedio* ( -k1* ((l10 + x1)* (sym.cos(theta) - sym.sin(theta)) )**2 ) # mal
pot_k1 = unMedio* k1* (l10 + x1)**2 # Lo escribi yo
# pot_k2 = unMedio* -k2* (l20 + x)**2
pot_k2 = unMedio* k2* (l20 + x)**2
V = sym.Eq(sym.Symbol('V'), m1_V + pot_k1 + pot_k2 ) #agrega el potencial elastico k en la ecuacion
V
```

$$V = gm_1(-l_{10} - x_1) \sin(\theta) + \frac{k_1(l_{10} + x_1)^2}{2} + \frac{k_2(l_{20} + x)^2}{2}$$

▼ Lagrangiano

```
[ ] L = sym.Eq(sym.Symbol('\mathcal{L}'), (T.rhs - V.rhs))
L
```

$$\mathcal{L} = -gm_1(-l_{10} - x_1) \sin(\theta) - \frac{k_1(l_{10} + x_1)^2}{2} - \frac{k_2(l_{20} + x)^2}{2} + \frac{(m_0 + m_1)(2 \cos(\theta) \dot{x}_1 + \dot{x}^2 + \dot{x}_1^2)}{2}$$

ECUACIONES DE EULER

Para x

Comentar

Compartir

Editar

Conectar

Editando

Victor Alexis Bettachini

31 de may. de 2021

(editado el 31 de may. de 2021)

[Resolver](#)

- El estiramiento del resorte de k_1 es colineal con x_1 . No tienen sentido pensar en proyecciones (si es lo que hiciste, que realmente no entiendo).
- ¿Porque negativos los k ?

Victor Alexis Bettachini

31 de may. de 2021

[Resolver](#)

Repositorio GitHub

- Cuaderno Jupyter: GitHub $\xrightarrow{\text{carga}}$ Google Colaboratory

The screenshot shows the GitHub interface for the repository 'MecanicaAnaliticaComputacional' by user 'unlam'. The repository is public and has 1 watch, 1 fork, and 0 stars. The main branch is 'main'. The repository contains a table of files and folders, including '.idx', '.vscode', '01Vectorial', '02Energia', '03EulerLagrange', '04Ligaduras', '05Simulación', '06FuerzasLigadura', and '07NoConservativas'. The 'About' section describes the repository as a repository for the course 'Mecánica Analítica Computacional' from the degree in mechanical engineering at the Universidad Nacional de La Matanza. The 'Releases' section shows 1 tag.

unlam / **MecanicaAnaliticaComputacional**

Code Issues Pull requests Actions Projects Wiki Security Insights Settings

MecanicaAnaliticaComputacional Public Edit Pins Watch 1 Fork 1 Star 0

main 1 Branch 1 Tag Go to file + <> Code

bettachini constantes ✓ 338ed1c · 2 weeks ago 263 Commits

.idx	nix conf for idx	last year
.vscode	07NoConservativas/guiaNoConservativas	4 months ago
01Vectorial	constantes	2 weeks ago
02Energia	constantes	2 weeks ago
03EulerLagrange	08TensorInercia/monóxidoCarbono_tensorInercia.ip...	2 months ago
04Ligaduras	alles 2	2 months ago
05Simulación	alles 2	2 months ago
06FuerzasLigadura	sistemasNoHolónomos referencia Lanczos 2	2 months ago
07NoConservativas	07NoConservativas/noConservativas.ipynb_5	2 weeks ago

About

Repositorio de la asignatura "Mecánica Analítica Computacional" de la carrera de grado en ingeniería mecánica de la Universidad Nacional de La Matanza.

Readme View license Activity Custom properties 0 stars 1 watching 1 fork Report repository

Releases

1 tags

3er clase — SymPy: álgebra y análisis simbólico

Ecuaciones de Euler-Lagrange

Para x

```
[8]: x_EL = sym.Eq(L.rhs.diff(x) - L.rhs.diff(x.diff(t)).diff(t), 0).simplify() # ecuación igualando a cero
x_EL
```

$$[8]: m_1 \ddot{x} + m_2 \left(-\ell \sin(\phi) \dot{\phi}^2 + \ell \cos(\phi) \ddot{\phi} + \ddot{x} \right) = 0$$

Esta es una ecuación diferencial lineal de segundo orden homogénea. De aquí podría despejarse $\ddot{x}$

```
[9]: sym.Eq(x.diff(t,2),
          list( sym.solve(x_EL, x.diff(t,2) ) )[0] # solveset devuelve un set, que convertimos a lista
          ) # aceleración = x punto punto [m s-2]
```

$$[9]: \ddot{x} = \frac{\ell m_2 \left(\sin(\phi) \dot{\phi}^2 - \cos(\phi) \ddot{\phi} \right)}{m_1 + m_2}$$

Pero queda en función de otra aceleración $\ddot{\phi}$.

Para ϕ

```
[10]: phi_EL = sym.Eq(L.rhs.diff(phi) - L.rhs.diff(phi.diff(t)).diff(t), 0).simplify() # ecuación igualando a cero
phi_EL
```

5ta clase — SciPy: cálculo numérico

```
[22]: # defino una función con el sistema de derivadas
      # t : no se usa en este sistema pero lo dejamos para uso posterior
      # y : lista de estado con [y[0], y[1], y[2], y[3]]
      # y[0]: x
      # y[1]: x punto
      # y[2]: phi
      # y[3]: phi punto
      # dydt : lista de derivadas
      def y_punto(t, y):
          dydt = [y[1],
                  x_pp_numpy(y[0], y[1], y[2], y[3]),
                  y[3],
                  phi_pp_numpy(y[0], y[1], y[2], y[3]),
                  ]
          return dydt

[23]: # Integración de a pasos en el tiempo
      y_ode2 = solve_ivp(y_punto, (t_rango[0], t_rango[-1]), y_inicial, t_eval = t_rango)

[25]: y_ode2.y[0]
```

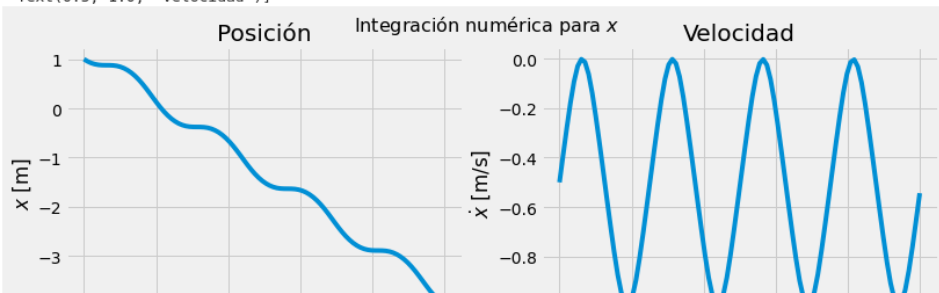
```
[25]: array([[ 1.          ,  0.95510744,  0.92131146,  0.89820932,  0.88468059,
            0.87877042,  0.87745354,  0.87702754,  0.87352768,  0.86357726,
            0.84474673,  0.81565733,  0.77559949,  0.72423163,  0.66166451,
            0.588266   ,  0.50468237,  0.41250381,  0.31433661,  0.21366454,
            0.11444308,  0.02023394, -0.06599563, -0.14244216, -0.20809592,
            -0.26250272, -0.30576388, -0.33796804, -0.35953138, -0.37175469,
            -0.3760222 , -0.37701011, -0.37854158, -0.38007486, -0.38107026])
```

5ta clase — matplotlib: análisis gráfico de resultados • • • • •

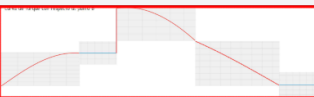
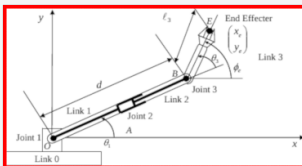
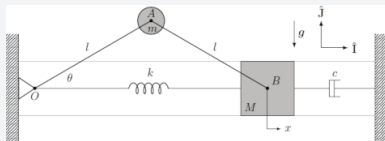
```
[26]: solucion = y_ode2
      nombreCoordenada = 'x'

      fig, ax = plt.subplots(nrows= 1, ncols= 2, squeeze=False, figsize=(12, 4)) # dos figuras en la misma fila
      fig.suptitle('Integración numérica para $'+ nombreCoordenada + '$', fontsize=16)
      ax[0,0].plot(solucion.t, solucion.y[0]) # posición x
      ax[0,0].set(xlabel='t [s]', ylabel= '$' + nombreCoordenada+ '$ [m]', title='Posición')
      ax[0,1].plot(solucion.t, solucion.y[1]) # velocidad x
      ax[0,1].set(xlabel='t [s]', ylabel='$\dot{x}$ [m/s]', title='Velocidad')
```

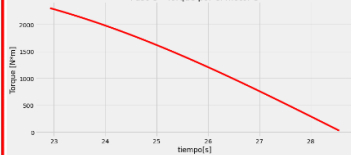
```
[26]: [Text(0.5, 0, 't [s]'),
      Text(0, 0.5, '$\dot{x}$ [m/s]'),
      Text(0.5, 1.0, 'Velocidad')]
```



7ma clase — Esfuerzos y dinámica de máquinas



Paso 5 - torque por el motor 3



Paso 6

En este paso la primera sección se extiende según los valores calculados durante el desarrollo del paso 4. L2 se extiende unos 1.0778 metros, calculamos el tiempo transcurrido para esto.

```
tp6 = l2f/v_v  
tpp6 = sym.symbols('Tiempo_paso6')  
sym.Eq(tpp6, tp6) #segundos
```

Tiempo_{paso6} = 21.5571741279056

Motor 1

```
Qbr1p6 = nc11*g #Fuerza para el cilindro 1  
Qbr2p6 = nc11*g #Fuerza para el cilindro 2  
Qbr3p6 = nc11*g #Fuerza para el cilindro 3  
Qep6 = (m*mp)*g #Fuerza para el conjunto efector-pieza  
Qp6 = 3*Qbr1p6 + Qep6 #Fuerza total realizada por el primer motor motor 1
```

```
Wbr1p6 = Qbr1p6*y1.diff(12) #Torque para el cilindro 1  
Wbr2p6 = Qbr2p6*y2p3.diff(12) #Torque para el cilindro 2  
Wbr3p6 = Qbr3p6*y3p5.diff(12) #Torque para el cilindro 3  
Wep6 = Qep6*yep5.diff(12) #Torque para el conjunto efector-pieza  
Wp6 = Wbr1p6 + Wbr2p6 + Wbr3p6 + Wep6 #Torque total realizada por el motor 1
```

8va clase — GitHub Copilot: código de un LLM

- Tras dominar lo básico los estudiantes pueden aprovecharse de la IA.

```
lagrangiano = (T.rhs - V.rhs).expand()
t = sym.Symbol('t') # como se deriva respecto al tiempo con la función diff se declara t como símbolo
return sym.Eq(
    lagrangiano.diff(coordenadaGeneralizada)
    - lagrangiano.diff(coordenadaGeneralizada.diff(t)).diff(t)
    , 0
).simplify()
```

[21]

```
x1_EL = eulerLagrange(T, V, x1)
x1_EL
```

[22]

...

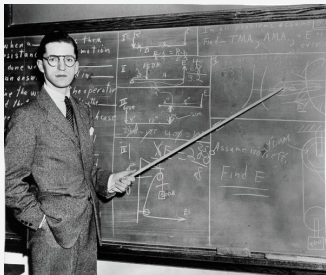
$$\frac{\pi^2 M \ddot{x}_1}{2} - gm_1 + gm_2 + m_1 \ddot{x}_1 + m_2 \ddot{x}_1 = 0$$

Esta es una ecuación diferencial lineal de segundo orden homogénea. De aquí se puede despejar $\ddot{x}$

▷

```
#Despejar x1PuntoPunto
x1PuntoPunto = sym.solve(x1_EL, x1.diff(t, t)).args[0]
```

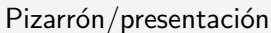
Valorar el tiempo de docentes y alumnos



Pizarrón/presentación

- Profesor: prepara lecciones $\xrightarrow{\text{de memoria}}$ pizarrón/presentación
- Estudiante: pizarrón/presentación $\xrightarrow{\text{transcribe}}$ cuaderno
- Práctica: **rehacer** los mismo cálculos, diagramas, etc. . .
- Lento y reiterativo $\implies \downarrow$ interés en la asignatura

● ● ● ● ● ● ●



- Licklider (1957): 85 % de “pensar” es lo “mundano”: organizar información, calcular, dibujar...

Cuaderno Jupyter/repositorio GitHub

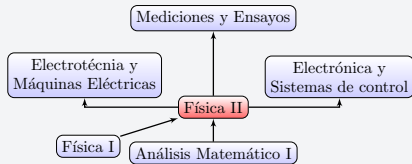
- Profesor: ideas $\xrightarrow{\text{actualiza}}$ código/teoría en repositorio
- Estudiante: repositorio del curso $\xrightarrow{\text{clona}}$ uno propio
- Práctica: **reciclar** código para resolver nuevos problemas

Temario

- 
- 
- 1 Herramientas que definen el aula
 - 2 UNLaM — Mecánica analítica
 - 3 UTN-FRA — Física II**
 - 4 Para concluir

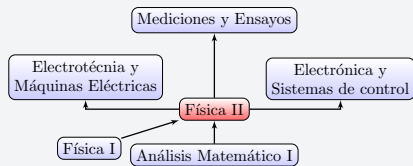
El problema inverso: insuficiente matemática

Física II en UTN-FRA



El problema inverso: insuficiente matemática

Física II en UTN-FRA



Salvando limitaciones de los alumnos

- Carecen del análisis para estudiar distribuciones de carga eléctrica → aproximado con discretas.
- Sin programación o cálculo numérico → funciones encapsuladas.
- Plantilla Jupyter para completar → estudio de casos inalcanzables con su matemática.

Líneas de campo de anillo cargado

Posición de cargas

```
Q = [
[0.013, 0.200, 0.006, 0],
[0.012, 0.199, 0.019, 0],
[0.012, 0.198, 0.031, 0],
[0.011, 0.195, 0.044, 0],
[0.011, 0.192, 0.056, 0],
[0.010, 0.188, 0.068, 0],
[0.009, 0.184, 0.079, 0],
#...
[0.011, 0.192, -0.056, 0],
[0.011, 0.195, -0.044, 0],
[0.012, 0.198, -0.031, 0],
[0.012, 0.199, -0.019, 0],
[0.013, 0.200, -0.006, 0]]
```

✓ 0.0s

Python

Cálculo del campo

```
# Definimos una función para la densidad:
def Lambda(t):
    a = 1E-9
    return a*cos(2*t)

R = 20E-2 # Radio del anillo en m.
N = 1000 # Número de cargas en que se divide el objeto.

dt = 2*pi/N # Separación angular entre cargas.
t0 = dt/2 # Angulo de la primera carga.
Q = []
ds = R*dt # Longitud de arco para cada segmento de anillo.
for l in np.arange(N):
    t = t0 + l*dt
    dq = Lambda(t)*ds # Carga de un pequeño arco de anillo.
    Q = Q + [[dq, R*cos(t0+l*dt), R*sin(t0+l*dt), 0]]
```

Python

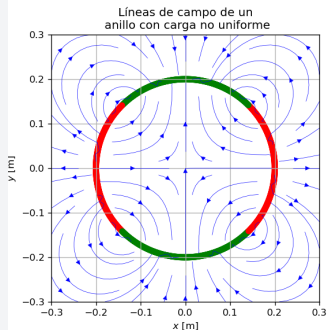
```
titulo = 'Líneas de campo de un\n anillo con carga\n no uniforme'
```

```
plotEf(Q, dx=0.3, title=titulo)
```

✓ 4.5s

Python

Líneas de campo



Saliendo del plano: dipolo eléctrico

Curvas equipotenciales

```
# Formato para la lista de cargas,  
# q en Coulombs, x,y,z en metros:  
# [  
# [q1, x1, y1, z1]  
# [q2, x2, y2, z2]  
# ]
```

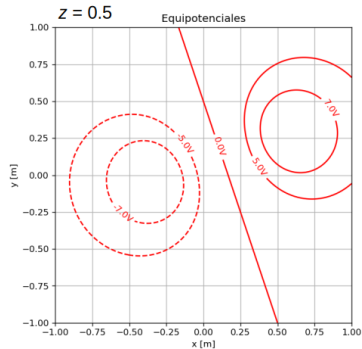
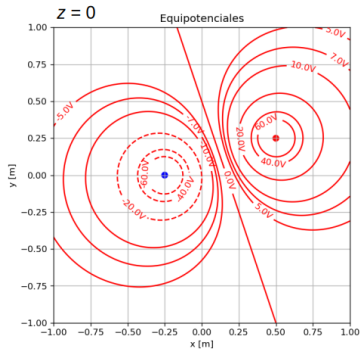
```
Q = [  
    [-1E-9, -0.25, 0, 0],  
    [ 1E-9, 0.5, 0.25, 0],  
]
```

```
niveles = [-60, -40, -20, -10, -7, -5, 0, 5, 7, 10,  
           20, 40, 60]
```

```
equipotencialesPuntuales(Q, niveles=niveles, z = 0)
```

✓ 0.1s

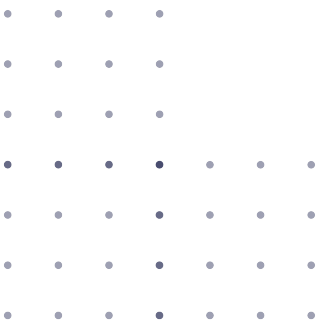
Python



Valoración de los alumnos de la experiencia

- Evita frustraciones: estudian problemas que podrían resolverlos analíticamente, pero fallarían mucho
- Genera interés: problemas realistas.
- Retroalimentación permite la prueba y error.
- Comprueban resultados de problemas símil examen.

Temario

- 
- 
- 1 Herramientas que definen el aula
 - 2 UNLaM — Mecánica analítica
 - 3 UTN-FRA — Física II
 - 4 Para concluir

Código en cursos de ingeniería mecánica

Objeciones

- “Los ingenieros mecánicos no programan.”
 - Modificar código no es programar.
 - Código: omnipresente en la industria.
- “No hay suficientes computadoras.”
 - Las hay en hogares y trabajos (pandemia).
 - Se probó en clase editar y ejecutar código en Google Colaboratory usando teléfonos.

Código en cursos de ingeniería mecánica

Objeciones

- “Los ingenieros mecánicos no programan.”
 - Modificar código no es programar.
 - Código: omnipresente en la industria.
- “No hay suficientes computadoras.”
 - Las hay en hogares y trabajos (pandemia).
 - Se probó en clase editar y ejecutar código en Google Colaboratory usando teléfonos.

Código en el aula

- Análisis casos realistas.
- Del docente conferenciante → mentor ante ejercicios.
- Eficientizar tiempo: copiar en papel → reciclar código.
- Colaboración remota.
- Asistencia de IA.

**MUCHAS
GRACIAS**

